

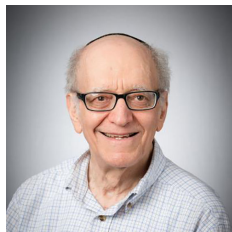
WhileCC-approximability and Acceptability of Elementary Functions or The story of the time I invented a different kind of Wheel

Fateme Ghasemi Jeffery Zucker William F. Smyth
ghases5@mcmaster.ca zucker@mcmaster.ca smyth@mcmaster.ca

CiE 2026
Trier University
July 27th - 31st, 2027



In loving memory of Prof. Jeff Zucker



Overview

- Problem
- Key Insights from Original Proof
- A Simpler Approach

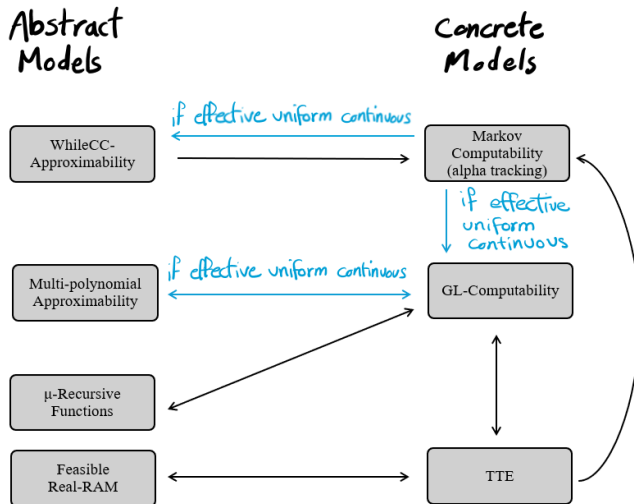
Overview

- Problem
- Key Insights from Original Proof
- A Simpler Approach

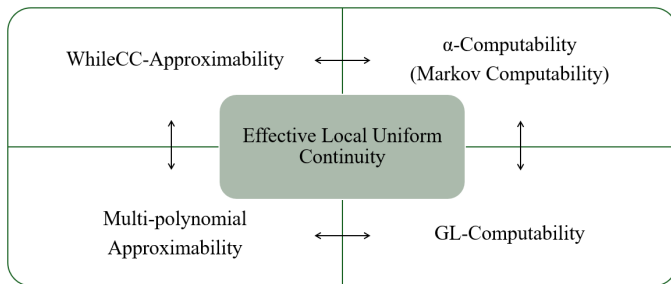
Overview

- Problem
- Key Insights from Original Proof
- A Simpler Approach

Computability of Total Functions on $\mathbb{R}$



Computability of Total Functions on $\mathbb{R}$

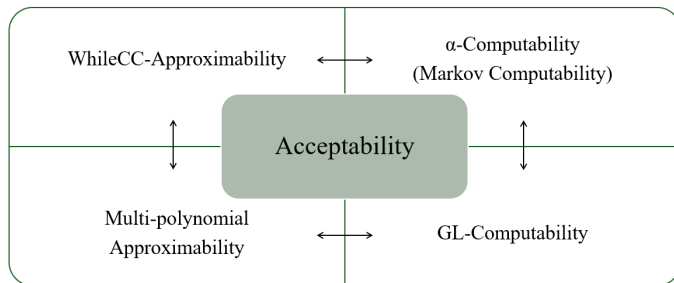


Computability of Partial Functions on $\mathbb{R}$

For partial functions on $\mathbb{R}$, Fu and Zucker [2014] generalize effectively locally uniform continuity to **acceptability** to get an equivalence.

Problem

How general is this class of acceptable functions?



Useful First Step Towards Solution

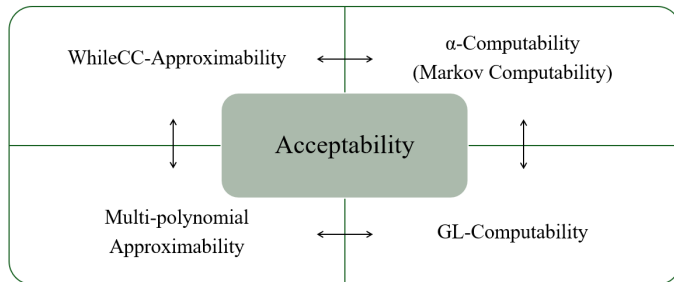
Show that the elementary functions satisfy the acceptability conditions.

Computability of Partial Functions on $\mathbb{R}$

For partial functions on $\mathbb{R}$, Fu and Zucker [2014] generalize effectively locally uniform continuity to **acceptability** to get an equivalence.

Problem

How general is this class of acceptable functions?



Useful First Step Towards Solution

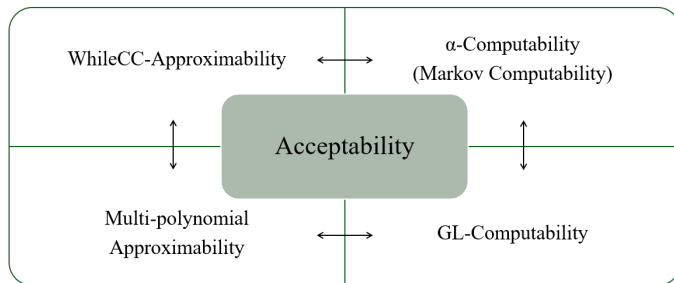
Show that the elementary functions satisfy the acceptability conditions.

Computability of Partial Functions on $\mathbb{R}$

For partial functions on $\mathbb{R}$, Fu and Zucker [2014] generalize effectively locally uniform continuity to **acceptability** to get an equivalence.

Problem

How general is this class of acceptable functions?



Useful First Step Towards Solution

Show that the elementary functions satisfy the acceptability conditions.

Background – Acceptability

Definition (**Acceptability**)

A function $f : \mathbb{R} \rightarrow \mathbb{R}$ is **acceptable** if there exists a sequence X where:

- 1 X is an **effective open exhaustion** for $\text{dom}(f)$, and
- 2 f is **effectively locally uniformly continuous w.r.t. X** .

Background – Acceptability

Definition (**Acceptability**)

A function $f : \mathbb{R} \rightarrow \mathbb{R}$ is **acceptable** if there exists a sequence X where:

- 1 X is an **effective open exhaustion** for $\text{dom}(f)$, and
- 2 f is **effectively locally uniformly continuous w.r.t. X** .

Background – Acceptability

Definition (**Acceptability**)

A function $f : \mathbb{R} \rightarrow \mathbb{R}$ is **acceptable** if there exists a sequence X where:

- 1 X is an **effective open exhaustion** for $\text{dom}(f)$, and
- 2 f is **effectively locally uniformly continuous w.r.t. X** .

Background – Effective Open Exhaustions

Definition ([Fu and Zucker, 2014])

A sequence $(U_1, U_2, \dots)$ of open sets is called an **effective open exhaustion** for an open $U \subseteq \mathbb{R}$ if

- 1 $U = \bigcup_{l=0}^{\infty} U_l$, and
- 2 for each $l \in \mathbb{N}$, U_l is a finite union of non-empty open finite intervals $I_1^l, I_2^l, \dots, I_{k_l}^l$ whose closures are pairwise disjoint, and
- 3 for each $l \in \mathbb{N}$, $\overline{U_l} = \bigcup_{i=1}^{k_l} \overline{I_i^l} \subseteq U_{l+1}$.
- 4 for all l , the components I_i^l that are intervals building up the stage U_l , are *rational* and *ordered* i.e., $I_i^l = (a_i^l, b_i^l)$ for some $a_i^l, b_i^l \in \mathbb{Q}$ where $b_i^l < a_{i+1}^l$ for $i = 1, \dots, k_l - 1$, and
- 5 the map $l \mapsto (a_1^l, b_1^l, \dots, a_{k_l}^l, b_{k_l}^l)$ which delivers the sequence of stages $U_l = I_1^l \cup \dots \cup I_{k_l}^l$ is recursive.

- The sequence of open sets $(-1, 1), (-2, 2), \dots, (-k, k), \dots$ is the standard effective open exhaustion for $\mathbb{R}$.
- The notion of effective open exhaustions is equivalent to the notion of **recursively enumerable (r.e./c.e) open sets** [Weihrauch, 2000].

Background – Effective Open Exhaustions

Definition ([Fu and Zucker, 2014])

A sequence $(U_1, U_2, \dots)$ of open sets is called an **effective open exhaustion** for an open $U \subseteq \mathbb{R}$ if

- 1 $U = \bigcup_{l=0}^{\infty} U_l$, and
- 2 for each $l \in \mathbb{N}$, U_l is a finite union of non-empty open finite intervals $I_1^l, I_2^l, \dots, I_{k_l}^l$ whose closures are pairwise disjoint, and
- 3 for each $l \in \mathbb{N}$, $\overline{U_l} = \bigcup_{i=1}^{k_l} \overline{I_i^l} \subseteq U_{l+1}$.
- 4 for all l , the components I_i^l that are intervals building up the stage U_l , are *rational* and *ordered* i.e., $I_i^l = (a_i^l, b_i^l)$ for some $a_i^l, b_i^l \in \mathbb{Q}$ where $b_i^l < a_{i+1}^l$ for $i = 1, \dots, k_l - 1$, and
- 5 the map $l \mapsto (a_1^l, b_1^l, \dots, a_{k_l}^l, b_{k_l}^l)$ which delivers the sequence of stages $U_l = I_1^l \cup \dots \cup I_{k_l}^l$ is recursive.

- The sequence of open sets $(-1, 1), (-2, 2), \dots, (-k, k), \dots$ is the standard effective open exhaustion for $\mathbb{R}$.
- The notion of effective open exhaustions is equivalent to the notion of **recursively enumerable (r.e./c.e) open sets** [Weihrauch, 2000].

Background – Effective Open Exhaustions

Definition ([Fu and Zucker, 2014])

A sequence $(U_1, U_2, \dots)$ of open sets is called an **effective open exhaustion** for an open $U \subseteq \mathbb{R}$ if

- ① $U = \bigcup_{l=0}^{\infty} U_l$, and
- ② for each $l \in \mathbb{N}$, U_l is a finite union of non-empty open finite intervals $I_1^l, I_2^l, \dots, I_{k_l}^l$ whose closures are pairwise disjoint, and
- ③ for each $l \in \mathbb{N}$, $\overline{U_l} = \bigcup_{i=1}^{k_l} \overline{I_i^l} \subseteq U_{l+1}$.
- ④ for all l , the components I_i^l that are intervals building up the stage U_l , are *rational* and *ordered* i.e., $I_i^l = (a_i^l, b_i^l)$ for some $a_i^l, b_i^l \in \mathbb{Q}$ where $b_i^l < a_{i+1}^l$ for $i = 1, \dots, k_l - 1$, and
- ⑤ the map $l \mapsto (a_1^l, b_1^l, \dots, a_{k_l}^l, b_{k_l}^l)$ which delivers the sequence of stages $U_l = I_1^l \cup \dots \cup I_{k_l}^l$ is recursive.

- The sequence of open sets $(-1, 1), (-2, 2), \dots, (-k, k), \dots$ is the standard effective open exhaustion for $\mathbb{R}$.
- The notion of effective open exhaustions is equivalent to the notion of **recursively enumerable (r.e./c.e) open sets** [Weihrauch, 2000].

Background – Effective Open Exhaustions

Definition ([Fu and Zucker, 2014])

A sequence $(U_1, U_2, \dots)$ of open sets is called an **effective open exhaustion** for an open $U \subseteq \mathbb{R}$ if

- ❶ $U = \bigcup_{l=0}^{\infty} U_l$, and
- ❷ for each $l \in \mathbb{N}$, U_l is a finite union of non-empty open finite intervals $I_1^l, I_2^l, \dots, I_{k_l}^l$ whose closures are pairwise disjoint, and
- ❸ for each $l \in \mathbb{N}$, $\overline{U_l} = \bigcup_{i=1}^{k_l} \overline{I_i^l} \subseteq U_{l+1}$.
- ❹ for all l , the components I_i^l that are intervals building up the stage U_l , are *rational* and *ordered* i.e., $I_i^l = (a_i^l, b_i^l)$ for some $a_i^l, b_i^l \in \mathbb{Q}$ where $b_i^l < a_{i+1}^l$ for $i = 1, \dots, k_l - 1$, and
- ❺ the map $l \mapsto (a_1^l, b_1^l, \dots, a_{k_l}^l, b_{k_l}^l)$ which delivers the sequence of stages $U_l = I_1^l \cup \dots \cup I_{k_l}^l$ is recursive.

- The sequence of open sets $(-1, 1), (-2, 2), \dots, (-k, k), \dots$ is the standard effective open exhaustion for $\mathbb{R}$.
- The notion of effective open exhaustions is equivalent to the notion of **recursively enumerable (r.e./c.e) open sets** [Weihrauch, 2000].

Background – Effective Open Exhaustions

Definition ([Fu and Zucker, 2014])

A sequence $(U_1, U_2, \dots)$ of open sets is called an **effective open exhaustion** for an open $U \subseteq \mathbb{R}$ if

- 1 $U = \bigcup_{l=0}^{\infty} U_l$, and
- 2 for each $l \in \mathbb{N}$, U_l is a finite union of non-empty open finite intervals $I_1^l, I_2^l, \dots, I_{k_l}^l$ whose closures are pairwise disjoint, and
- 3 for each $l \in \mathbb{N}$, $\overline{U_l} = \bigcup_{i=1}^{k_l} \overline{I_i^l} \subseteq U_{l+1}$.
- 4 for all l , the components I_i^l that are intervals building up the stage U_l , are *rational* and *ordered* i.e., $I_i^l = (a_i^l, b_i^l)$ for some $a_i^l, b_i^l \in \mathbb{Q}$ where $b_i^l < a_{i+1}^l$ for $i = 1, \dots, k_l - 1$, and
- 5 the map $l \mapsto (a_1^l, b_1^l, \dots, a_{k_l}^l, b_{k_l}^l)$ which delivers the sequence of stages $U_l = I_1^l \cup \dots \cup I_{k_l}^l$ is recursive.

- The sequence of open sets $(-1, 1), (-2, 2), \dots, (-k, k), \dots$ is the standard effective open exhaustion for $\mathbb{R}$.
- The notion of effective open exhaustions is equivalent to the notion of **recursively enumerable (r.e./c.e) open sets** [Weihrauch, 2000].

Background – Effective Open Exhaustions

Definition ([Fu and Zucker, 2014])

A sequence $(U_1, U_2, \dots)$ of open sets is called an **effective open exhaustion** for an open $U \subseteq \mathbb{R}$ if

- 1 $U = \bigcup_{l=0}^{\infty} U_l$, and
- 2 for each $l \in \mathbb{N}$, U_l is a finite union of non-empty open finite intervals $I_1^l, I_2^l, \dots, I_{k_l}^l$ whose closures are pairwise disjoint, and
- 3 for each $l \in \mathbb{N}$, $\overline{U_l} = \bigcup_{i=1}^{k_l} \overline{I_i^l} \subseteq U_{l+1}$.
- 4 for all l , the components I_i^l that are intervals building up the stage U_l , are *rational* and *ordered* i.e., $I_i^l = (a_i^l, b_i^l)$ for some $a_i^l, b_i^l \in \mathbb{Q}$ where $b_i^l < a_{i+1}^l$ for $i = 1, \dots, k_l - 1$, and
- 5 the map $l \mapsto (a_1^l, b_1^l, \dots, a_{k_l}^l, b_{k_l}^l)$ which delivers the sequence of stages $U_l = I_1^l \cup \dots \cup I_{k_l}^l$ is recursive.

- The sequence of open sets $(-1, 1), (-2, 2), \dots, (-k, k), \dots$ is the standard effective open exhaustion for $\mathbb{R}$.
- The notion of effective open exhaustions is equivalent to the notion of **recursively enumerable (r.e./c.e) open sets** [Weihrauch, 2000].

Background – Effective Open Exhaustions

Definition ([Fu and Zucker, 2014])

A sequence $(U_1, U_2, \dots)$ of open sets is called an **effective open exhaustion** for an open $U \subseteq \mathbb{R}$ if

- 1 $U = \bigcup_{l=0}^{\infty} U_l$, and
- 2 for each $l \in \mathbb{N}$, U_l is a finite union of non-empty open finite intervals $I_1^l, I_2^l, \dots, I_{k_l}^l$ whose closures are pairwise disjoint, and
- 3 for each $l \in \mathbb{N}$, $\overline{U_l} = \bigcup_{i=1}^{k_l} \overline{I_i^l} \subseteq U_{l+1}$.
- 4 for all l , the components I_i^l that are intervals building up the stage U_l , are *rational* and *ordered* i.e., $I_i^l = (a_i^l, b_i^l)$ for some $a_i^l, b_i^l \in \mathbb{Q}$ where $b_i^l < a_{i+1}^l$ for $i = 1, \dots, k_l - 1$, and
- 5 the map $l \mapsto (a_1^l, b_1^l, \dots, a_{k_l}^l, b_{k_l}^l)$ which delivers the sequence of stages $U_l = I_1^l \cup \dots \cup I_{k_l}^l$ is recursive.

- The sequence of open sets $(-1, 1), (-2, 2), \dots, (-k, k), \dots$ is the standard effective open exhaustion for $\mathbb{R}$.
- The notion of effective open exhaustions is equivalent to the notion of **recursively enumerable (r.e./c.e) open sets** [Weihrauch, 2000].

Background – Effective Open Exhaustions

Definition ([Fu and Zucker, 2014])

A sequence $(U_1, U_2, \dots)$ of open sets is called an **effective open exhaustion** for an open $U \subseteq \mathbb{R}$ if

- ❶ $U = \bigcup_{l=0}^{\infty} U_l$, and
- ❷ for each $l \in \mathbb{N}$, U_l is a finite union of non-empty open finite intervals $I_1^l, I_2^l, \dots, I_{k_l}^l$ whose closures are pairwise disjoint, and
- ❸ for each $l \in \mathbb{N}$, $\overline{U_l} = \bigcup_{i=1}^{k_l} \overline{I_i^l} \subseteq U_{l+1}$.
- ❹ for all l , the components I_i^l that are intervals building up the stage U_l , are *rational* and *ordered* i.e., $I_i^l = (a_i^l, b_i^l)$ for some $a_i^l, b_i^l \in \mathbb{Q}$ where $b_i^l < a_{i+1}^l$ for $i = 1, \dots, k_l - 1$, and
- ❺ the map $l \mapsto (a_1^l, b_1^l, \dots, a_{k_l}^l, b_{k_l}^l)$ which delivers the sequence of stages $U_l = I_1^l \cup \dots \cup I_{k_l}^l$ is recursive.

- The sequence of open sets $(-1, 1), (-2, 2), \dots, (-k, k), \dots$ is the standard effective open exhaustion for $\mathbb{R}$.
- The notion of effective open exhaustions is equivalent to the notion of **recursively enumerable (r.e./c.e) open sets** [Weihrauch, 2000].

Background – Effective Open Exhaustions

Definition ([Fu and Zucker, 2014])

A sequence $(U_1, U_2, \dots)$ of open sets is called an **effective open exhaustion** for an open $U \subseteq \mathbb{R}$ if

- ❶ $U = \bigcup_{l=0}^{\infty} U_l$, and
- ❷ for each $l \in \mathbb{N}$, U_l is a finite union of non-empty open finite intervals $I_1^l, I_2^l, \dots, I_{k_l}^l$ whose closures are pairwise disjoint, and
- ❸ for each $l \in \mathbb{N}$, $\overline{U_l} = \bigcup_{i=1}^{k_l} \overline{I_i^l} \subseteq U_{l+1}$.
- ❹ for all l , the components I_i^l that are intervals building up the stage U_l , are *rational* and *ordered* i.e., $I_i^l = (a_i^l, b_i^l)$ for some $a_i^l, b_i^l \in \mathbb{Q}$ where $b_i^l < a_{i+1}^l$ for $i = 1, \dots, k_l - 1$, and
- ❺ the map $l \mapsto (a_1^l, b_1^l, \dots, a_{k_l}^l, b_{k_l}^l)$ which delivers the sequence of stages $U_l = I_1^l \cup \dots \cup I_{k_l}^l$ is recursive.

- The sequence of open sets $(-1, 1), (-2, 2), \dots, (-k, k), \dots$ is the standard effective open exhaustion for $\mathbb{R}$.
- The notion of effective open exhaustions is equivalent to the notion of **recursively enumerable (r.e./c.e) open sets** [Weihrauch, 2000].

Background – Effective Open Exhaustions

Definition ([Fu and Zucker, 2014])

A sequence $(U_1, U_2, \dots)$ of open sets is called an **effective open exhaustion** for an open $U \subseteq \mathbb{R}$ if

- 1 $U = \bigcup_{l=0}^{\infty} U_l$, and
- 2 for each $l \in \mathbb{N}$, U_l is a finite union of non-empty open finite intervals $I_1^l, I_2^l, \dots, I_{k_l}^l$ whose closures are pairwise disjoint, and
- 3 for each $l \in \mathbb{N}$, $\overline{U_l} = \bigcup_{i=1}^{k_l} \overline{I_i^l} \subseteq U_{l+1}$.
- 4 for all l , the components I_i^l that are intervals building up the stage U_l , are *rational* and *ordered* i.e., $I_i^l = (a_i^l, b_i^l)$ for some $a_i^l, b_i^l \in \mathbb{Q}$ where $b_i^l < a_{i+1}^l$ for $i = 1, \dots, k_l - 1$, and
- 5 the map $l \mapsto (a_1^l, b_1^l, \dots, a_{k_l}^l, b_{k_l}^l)$ which delivers the sequence of stages $U_l = I_1^l \cup \dots \cup I_{k_l}^l$ is recursive.

- The sequence of open sets $(-1, 1), (-2, 2), \dots, (-k, k), \dots$ is the standard effective open exhaustion for $\mathbb{R}$.
- The notion of effective open exhaustions is equivalent to the notion of **recursively enumerable (r.e./c.e) open sets** [Weihrauch, 2000].

Background – Effective Open Exhaustions

Definition ([Fu and Zucker, 2014])

A sequence $(U_1, U_2, \dots)$ of open sets is called an **effective open exhaustion** for an open $U \subseteq \mathbb{R}$ if

- ❶ $U = \bigcup_{l=0}^{\infty} U_l$, and
- ❷ for each $l \in \mathbb{N}$, U_l is a finite union of non-empty open finite intervals $I_1^l, I_2^l, \dots, I_{k_l}^l$ whose closures are pairwise disjoint, and
- ❸ for each $l \in \mathbb{N}$, $\overline{U_l} = \bigcup_{i=1}^{k_l} \overline{I_i^l} \subseteq U_{l+1}$.
- ❹ for all l , the components I_i^l that are intervals building up the stage U_l , are *rational* and *ordered* i.e., $I_i^l = (a_i^l, b_i^l)$ for some $a_i^l, b_i^l \in \mathbb{Q}$ where $b_i^l < a_{i+1}^l$ for $i = 1, \dots, k_l - 1$, and
- ❺ the map $l \mapsto (a_1^l, b_1^l, \dots, a_{k_l}^l, b_{k_l}^l)$ which delivers the sequence of stages $U_l = I_1^l \cup \dots \cup I_{k_l}^l$ is recursive.

- The sequence of open sets $(-1, 1), (-2, 2), \dots, (-k, k), \dots$ is the standard effective open exhaustion for $\mathbb{R}$.
- The notion of effective open exhaustions is equivalent to the notion of **recursively enumerable (r.e./c.e) open sets** [Weihrauch, 2000].

Background – Effective Open Exhaustions

Definition ([Fu and Zucker, 2014])

A sequence $(U_1, U_2, \dots)$ of open sets is called an **effective open exhaustion** for an open $U \subseteq \mathbb{R}$ if

- ❶ $U = \bigcup_{l=0}^{\infty} U_l$, and
- ❷ for each $l \in \mathbb{N}$, U_l is a finite union of non-empty open finite intervals $I_1^l, I_2^l, \dots, I_{k_l}^l$ whose closures are pairwise disjoint, and
- ❸ for each $l \in \mathbb{N}$, $\overline{U_l} = \bigcup_{i=1}^{k_l} \overline{I_i^l} \subseteq U_{l+1}$.
- ❹ for all l , the components I_i^l that are intervals building up the stage U_l , are *rational* and *ordered* i.e., $I_i^l = (a_i^l, b_i^l)$ for some $a_i^l, b_i^l \in \mathbb{Q}$ where $b_i^l < a_{i+1}^l$ for $i = 1, \dots, k_l - 1$, and
- ❺ the map $l \mapsto (a_1^l, b_1^l, \dots, a_{k_l}^l, b_{k_l}^l)$ which delivers the sequence of stages $U_l = I_1^l \cup \dots \cup I_{k_l}^l$ is recursive.

- The sequence of open sets $(-1, 1), (-2, 2), \dots, (-k, k), \dots$ is the standard effective open exhaustion for $\mathbb{R}$.
- The notion of effective open exhaustions is equivalent to the notion of **recursively enumerable (r.e./c.e) open sets** [Weihrauch, 2000].

Background – Effective Local Uniform Continuity

Definition ([Fu and Zucker, 2014])

A function f on U is **effectively locally uniformly continuous w.r.t. an effective open exhaustion** $(U_n)_{n \in \mathbb{N}}$ of U , if there is a recursive function $M : \mathbb{N}^2 \rightarrow \mathbb{N}$ such that for all $k, l \in \mathbb{N}$ and all $x, y \in U_l$:

$$|x - y| < 2^{-M(k,l)} \implies |f(x) - f(y)| < 2^{-k}$$

Background – Effective Local Uniform Continuity

Definition ([Fu and Zucker, 2014])

A function f on U is **effectively locally uniformly continuous w.r.t. an effective open exhaustion** $(U_n)_{n \in \mathbb{N}}$ of U , if there is a recursive function $M : \mathbb{N}^2 \rightarrow \mathbb{N}$ such that for all $k, l \in \mathbb{N}$ and all $x, y \in U_l$:

$$|x - y| < 2^{-M(k,l)} \implies |f(x) - f(y)| < 2^{-k}$$

Background – Effective Local Uniform Continuity

Definition ([Fu and Zucker, 2014])

A function f on U is **effectively locally uniformly continuous w.r.t. an effective open exhaustion** $(U_n)_{n \in \mathbb{N}}$ of U , if there is a recursive function $M : \mathbb{N}^2 \rightarrow \mathbb{N}$ such that for all $k, l \in \mathbb{N}$ and all $x, y \in U_l$:

$$|x - y| < 2^{-M(k,l)} \implies |f(x) - f(y)| < 2^{-k}$$

Background – Elementary Functions

Definition ([Tenenbaum and Pollard, 1985])

The **elementary functions** on $\mathbb{R}$ are partial functions defined by expressions built up from

- computable reals, and
- the variable x ,

by applying (repeatedly) the basic operations below on elementary functions f, g :

- $(f + g)(x) = f(x) + g(x)$
- $(f \cdot g)(x) = f(x)g(x)$
- $\text{inv}_f(x) = \frac{1}{f(x)}$ where $\frac{1}{0} = \uparrow$
- $\text{root}_{n,f}(x) = \sqrt[n]{f(x)}$ where $0 < n \in \mathbb{N}$
- $\ln_f(x) = \ln(f(x))$
- $\exp_f(x) = e^{f(x)}$
- $\sin_f(x) = \sin(f(x))$
- $\arcsin_f(x) = \arcsin(f(x))$

Problem

The domains of elementary functions are not all open!

Solution: Modifications

- We define $\sqrt[n]{x} = 0$ for $x < 0$ when n is even.
- We extend the definition of $\arcsin(x)$ to be $\frac{\pi}{2}$ for $x > 1$ and to be $-\frac{\pi}{2}$ for $x < -1$.

Background – Elementary Functions

Definition ([Tenenbaum and Pollard, 1985])

The **elementary functions** on $\mathbb{R}$ are partial functions defined by expressions built up from

- computable reals, and
- the variable x ,

by applying (repeatedly) the basic operations below on elementary functions f, g :

- $(f + g)(x) = f(x) + g(x)$
- $(f \cdot g)(x) = f(x)g(x)$
- $\text{inv}_f(x) = \frac{1}{f(x)}$ where $\frac{1}{0} = \uparrow$
- $\text{root}_{n,f}(x) = \sqrt[n]{f(x)}$ where $0 < n \in \mathbb{N}$
- $\ln_f(x) = \ln(f(x))$
- $\exp_f(x) = e^{f(x)}$
- $\sin_f(x) = \sin(f(x))$
- $\arcsin_f(x) = \arcsin(f(x))$

Problem

The domains of elementary functions are not all open!

Solution: Modifications

- We define $\sqrt[n]{x} = 0$ for $x < 0$ when n is even.
- We extend the definition of $\arcsin(x)$ to be $\frac{\pi}{2}$ for $x > 1$ and to be $-\frac{\pi}{2}$ for $x < -1$.

Background – Elementary Functions

Definition ([Tenenbaum and Pollard, 1985])

The **elementary functions** on $\mathbb{R}$ are partial functions defined by expressions built up from

- computable reals, and
- the variable x ,

by applying (repeatedly) the basic operations below on elementary functions f, g :

- $(f + g)(x) = f(x) + g(x)$
- $(f \cdot g)(x) = f(x)g(x)$
- $\text{inv}_f(x) = \frac{1}{f(x)}$ where $\frac{1}{0} = \uparrow$
- $\text{root}_{n,f}(x) = \sqrt[n]{f(x)}$ where $0 < n \in \mathbb{N}$
- $\ln_f(x) = \ln(f(x))$
- $\exp_f(x) = e^{f(x)}$
- $\sin_f(x) = \sin(f(x))$
- $\arcsin_f(x) = \arcsin(f(x))$

Problem

The domains of elementary functions are not all open!

Solution: Modifications

- We define $\sqrt[n]{x} = 0$ for $x < 0$ when n is even.
- We extend the definition of $\arcsin(x)$ to be $\frac{\pi}{2}$ for $x > 1$ and to be $-\frac{\pi}{2}$ for $x < -1$.

Background – Elementary Functions

Definition ([Tenenbaum and Pollard, 1985])

The **elementary functions** on $\mathbb{R}$ are partial functions defined by expressions built up from

- computable reals, and
- the variable x ,

by applying (repeatedly) the basic operations below on elementary functions f, g :

- $(f + g)(x) = f(x) + g(x)$
- $(f \cdot g)(x) = f(x)g(x)$
- $\text{inv}_f(x) = \frac{1}{f(x)}$ where $\frac{1}{0} = \uparrow$
- $\text{root}_{n,f}(x) = \sqrt[n]{f(x)}$ where $0 < n \in \mathbb{N}$
- $\ln_f(x) = \ln(f(x))$
- $\exp_f(x) = e^{f(x)}$
- $\sin_f(x) = \sin(f(x))$
- $\arcsin_f(x) = \arcsin(f(x))$

Problem

The domains of elementary functions are not all open!

Solution: Modifications

- We define $\sqrt[n]{x} = 0$ for $x < 0$ when n is even.
- We extend the definition of $\arcsin(x)$ to be $\frac{\pi}{2}$ for $x > 1$ and to be $-\frac{\pi}{2}$ for $x < -1$.

Background – Elementary Functions

Definition ([Tenenbaum and Pollard, 1985])

The **elementary functions** on $\mathbb{R}$ are partial functions defined by expressions built up from

- computable reals, and
- the variable x ,

by applying (repeatedly) the basic operations below on elementary functions f, g :

- $(f + g)(x) = f(x) + g(x)$
- $(f \cdot g)(x) = f(x)g(x)$
- $\text{inv}_f(x) = \frac{1}{f(x)}$ where $\frac{1}{0} = \uparrow$
- $\text{root}_{n,f}(x) = \sqrt[n]{f(x)}$ where $0 < n \in \mathbb{N}$
- $\ln_f(x) = \ln(f(x))$
- $\exp_f(x) = e^{f(x)}$
- $\sin_f(x) = \sin(f(x))$
- $\arcsin_f(x) = \arcsin(f(x))$

Problem

The domains of elementary functions are not all open!

Solution: Modifications

- We define $\sqrt[n]{x} = 0$ for $x < 0$ when n is even.
- We extend the definition of $\arcsin(x)$ to be $\frac{\pi}{2}$ for $x > 1$ and to be $-\frac{\pi}{2}$ for $x < -1$.

Background – Elementary Functions

Definition ([Tenenbaum and Pollard, 1985])

The **elementary functions** on $\mathbb{R}$ are partial functions defined by expressions built up from

- computable reals, and
- the variable x ,

by applying (repeatedly) the basic operations below on elementary functions f, g :

- $(f + g)(x) = f(x) + g(x)$
- $(f \cdot g)(x) = f(x)g(x)$
- $\text{inv}_f(x) = \frac{1}{f(x)}$ where $\frac{1}{0} = \uparrow$
- $\text{root}_{n,f}(x) = \sqrt[n]{f(x)}$ where $0 < n \in \mathbb{N}$
- $\ln_f(x) = \ln(f(x))$
- $\exp_f(x) = e^{f(x)}$
- $\sin_f(x) = \sin(f(x))$
- $\arcsin_f(x) = \arcsin(f(x))$

Problem

The domains of elementary functions are not all open!

Solution: Modifications

- We define $\sqrt[n]{x} = 0$ for $x < 0$ when n is even.
- We extend the definition of $\arcsin(x)$ to be $\frac{\pi}{2}$ for $x > 1$ and to be $-\frac{\pi}{2}$ for $x < -1$.

Background – Elementary Functions

Definition ([Tenenbaum and Pollard, 1985])

The **elementary functions** on $\mathbb{R}$ are partial functions defined by expressions built up from

- computable reals, and
- the variable x ,

by applying (repeatedly) the basic operations below on elementary functions f, g :

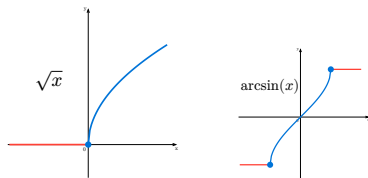
- $(f + g)(x) = f(x) + g(x)$
- $(f \cdot g)(x) = f(x)g(x)$
- $\text{inv}_f(x) = \frac{1}{f(x)}$ where $\frac{1}{0} = \uparrow$
- $\text{root}_{n,f}(x) = \sqrt[n]{f(x)}$ where $0 < n \in \mathbb{N}$
- $\ln_f(x) = \ln(f(x))$
- $\exp_f(x) = e^{f(x)}$
- $\sin_f(x) = \sin(f(x))$
- $\arcsin_f(x) = \arcsin(f(x))$

Problem

The domains of elementary functions are not all open!

Solution: Modifications

- We define $\sqrt[n]{x} = 0$ for $x < 0$ when n is even.
- We extend the definition of $\arcsin(x)$ to be $\frac{\pi}{2}$ for $x > 1$ and to be $-\frac{\pi}{2}$ for $x < -1$.



Background – Elementary Functions

Definition ([Tenenbaum and Pollard, 1985])

The **elementary functions** on $\mathbb{R}$ are partial functions defined by expressions built up from

- computable reals, and
- the variable x ,

by applying (repeatedly) the basic operations below on elementary functions f, g :

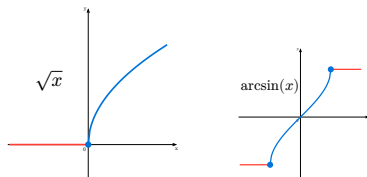
- $(f + g)(x) = f(x) + g(x)$
- $(f \cdot g)(x) = f(x)g(x)$
- $\text{inv}_f(x) = \frac{1}{f(x)}$ where $\frac{1}{0} = \uparrow$
- $\text{root}_{n,f}(x) = \sqrt[n]{f(x)}$ where $0 < n \in \mathbb{N}$
- $\ln_f(x) = \ln(f(x))$
- $\exp_f(x) = e^{f(x)}$
- $\sin_f(x) = \sin(f(x))$
- $\arcsin_f(x) = \arcsin(f(x))$

Problem

The domains of elementary functions are not all open!

Solution: Modifications

- We define $\sqrt[n]{x} = 0$ for $x < 0$ when n is even.
- We extend the definition of $\arcsin(x)$ to be $\frac{\pi}{2}$ for $x > 1$ and to be $-\frac{\pi}{2}$ for $x < -1$.



Background – Elementary Functions

Definition ([Tenenbaum and Pollard, 1985])

The **elementary functions** on $\mathbb{R}$ are partial functions defined by expressions built up from

- computable reals, and
- the variable x ,

by applying (repeatedly) the basic operations below on elementary functions f, g :

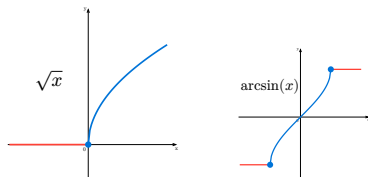
- $(f + g)(x) = f(x) + g(x)$
- $(f \cdot g)(x) = f(x)g(x)$
- $\text{inv}_f(x) = \frac{1}{f(x)}$ where $\frac{1}{0} = \uparrow$
- $\text{root}_{n,f}(x) = \sqrt[n]{f(x)}$ where $0 < n \in \mathbb{N}$
- $\ln_f(x) = \ln(f(x))$
- $\exp_f(x) = e^{f(x)}$
- $\sin_f(x) = \sin(f(x))$
- $\arcsin_f(x) = \arcsin(f(x))$

Problem

The domains of elementary functions are not all open!

Solution: Modifications

- We define $\sqrt[n]{x} = 0$ for $x < 0$ when n is even.
- We extend the definition of $\arcsin(x)$ to be $\frac{\pi}{2}$ for $x > 1$ and to be $-\frac{\pi}{2}$ for $x < -1$.



Contributions

Recall: Equivalence Theorem, [Fu and Zucker, 2014]

For any **acceptable function** $f : \mathbb{R} \rightarrow \mathbb{R}$ and any effective open exhaustion X for $\text{dom}(f)$, the following are equivalent:

- f is an α -computable function.
- f is **WhileCC**-approximable.
- f is GL-computable w.r.t. X .
- f is effectively locally uniformly multipolynomially approximable w.r.t. X .

Theorem 1 (**WhileCC**-approximability Theorem)

All elementary functions are **WhileCC**-approximable.

Theorem 2 (Acceptability Theorem)

All elementary functions are acceptable.

Contributions

Recall: Equivalence Theorem, [Fu and Zucker, 2014]

For any **acceptable function** $f : \mathbb{R} \rightarrow \mathbb{R}$ and any effective open exhaustion X for $\text{dom}(f)$, the following are equivalent:

- f is an α -computable function.
- f is **WhileCC**-approximable.
- f is GL-computable w.r.t. X .
- f is effectively locally uniformly multipolynomially approximable w.r.t. X .

Theorem 1 (WhileCC-approximability Theorem)

All elementary functions are **WhileCC**-approximable.

Theorem 2 (Acceptability Theorem)

All elementary functions are acceptable.

Contributions

Recall: Equivalence Theorem, [Fu and Zucker, 2014]

For any **acceptable function** $f : \mathbb{R} \rightarrow \mathbb{R}$ and any effective open exhaustion X for $\text{dom}(f)$, the following are equivalent:

- f is an α -computable function.
- f is **WhileCC**-approximable.
- f is GL-computable w.r.t. X .
- f is effectively locally uniformly multipolynomially approximable w.r.t. X .

Theorem 1 (WhileCC-approximability Theorem)

All elementary functions are **WhileCC**-approximable.

Theorem 2 (Acceptability Theorem)

All elementary functions are acceptable.

Contributions

Recall: Equivalence Theorem, [Fu and Zucker, 2014]

For any acceptable function $f : \mathbb{R} \rightarrow \mathbb{R}$ and any effective open exhaustion X for $\text{dom}(f)$, the following are equivalent:

- f is an α -computable function.
- f is **WhileCC**-approximable.
- f is GL-computable w.r.t. X .
- f is effectively locally uniformly multipolynomially approximable w.r.t. X .

Theorem 1 (WhileCC-approximability Theorem)

All elementary functions are **WhileCC**-approximable.

Theorem 2.1 (Acceptability Theorem: Part 1)

The domain of any elementary function has an effective open exhaustion.

Theorem 2.2 (Acceptability Theorem: Part 2)

Any elementary function is effectively locally uniformly continuous w.r.t. an effective open exhaustion for its domain.

Result 1

Theorem 1 (**WhileCC**-approximability Theorem)

All elementary functions are **WhileCC**-approximable.

Result 1

Theorem 1 (**WhileCC**-approximability Theorem)

All elementary functions are **WhileCC**-approximable.

Result 1 - Background - WhileCC Programming Language

Syntax

Terms

$$t^s ::= x^s \mid F(t_1^{s_1}, \dots, t_m^{s_m})$$

Statements

$S ::=$

skip $\mid \bar{x} := \bar{t} \mid S_1 \ S_2$

$\mid$ if b then S_1 else S_2 fi

$\mid$ while b do S_0 od

$\mid n := \text{choose} (z : \text{nat}) : P(z, \bar{t})$

Procedures

$P ::= \text{proc } D \text{ begin } S \text{ end}$

Algebra $\mathcal{R}$

$0_{\mathbb{R}}, 1_{\mathbb{R}}, -1_{\mathbb{R}} : \rightarrow \mathbb{R}$
 $+_{\mathbb{R}}, \times_{\mathbb{R}} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$
 $+_{\mathbb{N}}, \times_{\mathbb{N}} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$
 $\text{inv}_{\mathbb{R}} : \mathbb{R} \rightarrow \mathbb{R}$
 $0_{\mathbb{N}} : \rightarrow \mathbb{N}$
 $\text{suc}_{\mathbb{N}} : \mathbb{N} \rightarrow \mathbb{N}$
 $\text{tt}, \text{ff} : \rightarrow \mathbb{B}$
 $\text{and}, \text{or} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$
 $\text{not} : \mathbb{B} \rightarrow \mathbb{B}$
 $=_{\mathbb{N}}, <_{\mathbb{N}} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{B}$
 $=_{\text{real}}, <_{\mathbb{R}} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{B}$

Semantics

$$\text{inv}_{\mathbb{R}}(x) = \begin{cases} 1/x & \text{if } x \neq 0 \\ \uparrow & \text{otherwise} \end{cases}$$

$$=_{\text{real}}(x, y) = \begin{cases} \text{ff} & \text{if } x \neq y \\ \uparrow & \text{otherwise} \end{cases}$$

$$<_{\mathbb{R}}(x, y) = \begin{cases} \text{tt} & \text{if } x < y \\ \text{ff} & \text{if } x > y \\ \uparrow & \text{if } x = y \end{cases}$$

The Semantics of a program P is denoted by a many-valued function $P^{\mathcal{R}}$.

Statement	Possible Values for n
$n := \text{choose} (k : \text{nat}) : k < 0$	$\{\uparrow\}$
$n := \text{choose} (k : \text{nat}) : \text{toReal}(k) = 0$	$\{\uparrow\}$
$n := \text{choose} (k : \text{nat}) : k < k + 1$	$\{0, 1, 2, \dots\}$
$n := \text{choose} (k : \text{nat}) : k > 2 \text{ and } k < 4$	$\{3\}$

Result 1 - Background - WhileCC Programming Language

Syntax

Terms

$$t^s ::= x^s \mid F(t_1^{s_1}, \dots, t_m^{s_m})$$

Statements

$S ::=$

skip $\mid \bar{x} := \bar{t} \mid S_1 \ S_2$

$\mid$ if b then S_1 else S_2 fi

$\mid$ while b do S_0 od

$\mid n := \text{choose } (z : \text{nat}) : P(z, \bar{t})$

Procedures

$P ::= \text{proc } D \text{ begin } S \text{ end}$

Algebra $\mathcal{R}$

$0_{\mathbb{R}}, 1_{\mathbb{R}}, -1_{\mathbb{R}} : \rightarrow \mathbb{R}$
 $+_{\mathbb{R}}, \times_{\mathbb{R}} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$
 $+_{\mathbb{N}}, \times_{\mathbb{N}} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$
 $\text{inv}_{\mathbb{R}} : \mathbb{R} \rightarrow \mathbb{R}$
 $0_{\mathbb{N}} : \rightarrow \mathbb{N}$
 $\text{suc}_{\mathbb{N}} : \mathbb{N} \rightarrow \mathbb{N}$
 $\text{tt}, \text{ff} : \rightarrow \mathbb{B}$
 $\text{and}, \text{or} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$
 $\text{not} : \mathbb{B} \rightarrow \mathbb{B}$
 $=_{\mathbb{N}}, <_{\mathbb{N}} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{B}$
 $=_{\text{real}}, <_{\mathbb{R}} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{B}$

Semantics

$$\text{inv}_{\mathbb{R}}(x) = \begin{cases} 1/x & \text{if } x \neq 0 \\ \uparrow & \text{otherwise} \end{cases}$$

$$=_{\text{real}}(x, y) = \begin{cases} \text{ff} & \text{if } x \neq y \\ \uparrow & \text{otherwise} \end{cases}$$

$$<_{\mathbb{R}}(x, y) = \begin{cases} \text{tt} & \text{if } x < y \\ \text{ff} & \text{if } x > y \\ \uparrow & \text{if } x = y \end{cases}$$

The Semantics of a program P is denoted by a many-valued function $P^{\mathcal{R}}$.

Statement	Possible Values for n
$n := \text{choose } (k : \text{nat}) : k < 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : \text{toReal}(k) = 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : k < k + 1$	$\{0, 1, 2, \dots\}$
$n := \text{choose } (k : \text{nat}) : k > 2 \text{ and } k < 4$	$\{3\}$

Result 1 - Background - WhileCC Programming Language

Syntax

• Terms

$$t^s ::= x^s \mid F(t_1^{s_1}, \dots, t_m^{s_m})$$

• Statements

$S ::=$

skip $\mid \bar{x} := \bar{t} \mid S_1 \ S_2$

$\mid$ if b then S_1 else S_2 fi

$\mid$ while b do S_0 od

$\mid n := \text{choose } (z : \text{nat}) : P(z, \bar{t})$

• Procedures

$P ::= \text{proc } D \text{ begin } S \text{ end}$

Algebra $\mathcal{R}$

$0_{\mathbb{R}}, 1_{\mathbb{R}}, -1_{\mathbb{R}} : \rightarrow \mathbb{R}$
 $+_{\mathbb{R}}, \times_{\mathbb{R}} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$
 $+_{\mathbb{N}}, \times_{\mathbb{N}} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$
 $\text{inv}_{\mathbb{R}} : \mathbb{R} \rightarrow \mathbb{R}$
 $0_{\mathbb{N}} : \rightarrow \mathbb{N}$
 $\text{suc}_{\mathbb{N}} : \mathbb{N} \rightarrow \mathbb{N}$
 $\text{tt}, \text{ff} : \rightarrow \mathbb{B}$
 $\text{and}, \text{or} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$
 $\text{not} : \mathbb{B} \rightarrow \mathbb{B}$
 $=_{\mathbb{N}}, <_{\mathbb{N}} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{B}$
 $=_{\text{real}}, <_{\mathbb{R}} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{B}$

Semantics

$$\text{inv}_{\mathbb{R}}(x) = \begin{cases} 1/x & \text{if } x \neq 0 \\ \uparrow & \text{otherwise} \end{cases}$$

$$=_{\text{real}}(x, y) = \begin{cases} \text{ff} & \text{if } x \neq y \\ \uparrow & \text{otherwise} \end{cases}$$

$$<_{\mathbb{R}}(x, y) = \begin{cases} \text{tt} & \text{if } x < y \\ \text{ff} & \text{if } x > y \\ \uparrow & \text{if } x = y \end{cases}$$

The Semantics of a program P is denoted by a many-valued function $P^{\mathcal{R}}$.

Statement	Possible Values for n
$n := \text{choose } (k : \text{nat}) : k < 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : \text{toReal}(k) = 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : k < k + 1$	$\{0, 1, 2, \dots\}$
$n := \text{choose } (k : \text{nat}) : k > 2 \text{ and } k < 4$	$\{3\}$

Result 1 - Background - WhileCC Programming Language

Syntax

Terms

$$t^s ::= x^s \mid F(t_1^{s_1}, \dots, t_m^{s_m})$$

Statements

$S ::=$

skip $\mid \bar{x} := \bar{t} \mid S_1 \ S_2$

$\mid$ if b then S_1 else S_2 fi

$\mid$ while b do S_0 od

$\mid n := \text{choose } (z : \text{nat}) : P(z, \bar{t})$

Procedures

$P ::= \text{proc } D \text{ begin } S \text{ end}$

Algebra $\mathcal{R}$

$0_{\mathbb{R}}, 1_{\mathbb{R}}, -1_{\mathbb{R}} : \rightarrow \mathbb{R}$
 $+_{\mathbb{R}}, \times_{\mathbb{R}} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$
 $+_{\mathbb{N}}, \times_{\mathbb{N}} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$
 $\text{inv}_{\mathbb{R}} : \mathbb{R} \rightarrow \mathbb{R}$
 $0_{\mathbb{N}} : \rightarrow \mathbb{N}$
 $\text{suc}_{\mathbb{N}} : \mathbb{N} \rightarrow \mathbb{N}$
 $\text{tt}, \text{ff} : \rightarrow \mathbb{B}$
 $\text{and}, \text{or} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$
 $\text{not} : \mathbb{B} \rightarrow \mathbb{B}$
 $=_{\mathbb{N}}, <_{\mathbb{N}} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{B}$
 $=_{\text{real}}, <_{\mathbb{R}} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{B}$

Semantics

$$\text{inv}_{\mathbb{R}}(x) = \begin{cases} 1/x & \text{if } x \neq 0 \\ \uparrow & \text{otherwise} \end{cases}$$

$$=_{\text{real}}(x, y) = \begin{cases} \text{ff} & \text{if } x \neq y \\ \uparrow & \text{otherwise} \end{cases}$$

$$<_{\mathbb{R}}(x, y) = \begin{cases} \text{tt} & \text{if } x < y \\ \text{ff} & \text{if } x > y \\ \uparrow & \text{if } x = y \end{cases}$$

The Semantics of a program P is denoted by a many-valued function $P^{\mathcal{R}}$.

Statement	Possible Values for n
$n := \text{choose } (k : \text{nat}) : k < 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : \text{toReal}(k) = 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : k < k + 1$	$\{0, 1, 2, \dots\}$
$n := \text{choose } (k : \text{nat}) : k > 2 \text{ and } k < 4$	$\{3\}$

Result 1 - Background - WhileCC Programming Language

Syntax

Terms

$$t^s ::= x^s \mid F(t_1^{s_1}, \dots, t_m^{s_m})$$

Statements

$S ::=$

skip $\mid \bar{x} := \bar{t} \mid S_1 \ S_2$

$\mid$ if b then S_1 else S_2 fi

$\mid$ while b do S_0 od

$\mid n := \text{choose } (z : \text{nat}) : P(z, \bar{t})$

Procedures

$P ::= \text{proc } D \text{ begin } S \text{ end}$

Algebra $\mathcal{R}$

$0_{\mathbb{R}}, 1_{\mathbb{R}}, -1_{\mathbb{R}} : \rightarrow \mathbb{R}$
 $+_{\mathbb{R}}, \times_{\mathbb{R}} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$
 $+_{\mathbb{N}}, \times_{\mathbb{N}} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$
 $\text{inv}_{\mathbb{R}} : \mathbb{R} \rightarrow \mathbb{R}$
 $0_{\mathbb{N}} : \rightarrow \mathbb{N}$
 $\text{suc}_{\mathbb{N}} : \mathbb{N} \rightarrow \mathbb{N}$
 $\text{tt}, \text{ff} : \rightarrow \mathbb{B}$
 $\text{and}, \text{or} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$
 $\text{not} : \mathbb{B} \rightarrow \mathbb{B}$
 $=_{\mathbb{N}}, <_{\mathbb{N}} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{B}$
 $=_{\text{real}}, <_{\mathbb{R}} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{B}$

Semantics

$$\text{inv}_{\mathbb{R}}(x) = \begin{cases} 1/x & \text{if } x \neq 0 \\ \uparrow & \text{otherwise} \end{cases}$$

$$=_{\text{real}}(x, y) = \begin{cases} \text{ff} & \text{if } x \neq y \\ \uparrow & \text{otherwise} \end{cases}$$

$$<_{\mathbb{R}}(x, y) = \begin{cases} \text{tt} & \text{if } x < y \\ \text{ff} & \text{if } x > y \\ \uparrow & \text{if } x = y \end{cases}$$

The Semantics of a program P is denoted by a many-valued function $P^{\mathcal{R}}$.

Statement	Possible Values for n
$n := \text{choose } (k : \text{nat}) : k < 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : \text{toReal}(k) = 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : k < k + 1$	$\{0, 1, 2, \dots\}$
$n := \text{choose } (k : \text{nat}) : k > 2 \text{ and } k < 4$	$\{3\}$

Result 1 - Background - WhileCC Programming Language

Syntax

• Terms

$$t^s ::= x^s \mid F(t_1^{s_1}, \dots, t_m^{s_m})$$

• Statements

$S ::=$

skip $\mid \bar{x} := \bar{t} \mid S_1 \ S_2$

$\mid$ if b then S_1 else S_2 fi

$\mid$ while b do S_0 od

$\mid n := \text{choose} (z : \text{nat}) : P(z, \bar{t})$

• Procedures

$P ::= \text{proc } D \text{ begin } S \text{ end}$

Algebra $\mathcal{R}$

$$\begin{aligned} 0_{\mathbb{R}}, 1_{\mathbb{R}}, -1_{\mathbb{R}} &: \rightarrow \mathbb{R} \\ +_{\mathbb{R}}, \times_{\mathbb{R}} &: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R} \\ +_{\mathbb{N}}, \times_{\mathbb{N}} &: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N} \\ \text{inv}_{\mathbb{R}} &: \mathbb{R} \rightarrow \mathbb{R} \\ 0_{\mathbb{N}} &: \rightarrow \mathbb{N} \\ \text{suc}_{\mathbb{N}} &: \mathbb{N} \rightarrow \mathbb{N} \\ \text{tt}, \text{ff} &: \rightarrow \mathbb{B} \\ \text{and}, \text{or} &: \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B} \\ \text{not} &: \mathbb{B} \rightarrow \mathbb{B} \\ =_{\mathbb{N}}, <_{\mathbb{N}} &: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{B} \\ =_{\text{real}}, <_{\mathbb{R}} &: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{B} \end{aligned}$$

Semantics

$$\text{inv}_{\mathbb{R}}(x) = \begin{cases} 1/x & \text{if } x \neq 0 \\ \uparrow & \text{otherwise} \end{cases}$$

$$=_{\text{real}}(x, y) = \begin{cases} \text{ff} & \text{if } x \neq y \\ \uparrow & \text{otherwise} \end{cases}$$

$$<_{\mathbb{R}}(x, y) = \begin{cases} \text{tt} & \text{if } x < y \\ \text{ff} & \text{if } x > y \\ \uparrow & \text{if } x = y \end{cases}$$

The Semantics of a program P is denoted by a many-valued function $P^{\mathcal{R}}$.

Statement	Possible Values for n
$n := \text{choose } (k : \text{nat}) : k < 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : \text{toReal}(k) = 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : k < k + 1$	$\{0, 1, 2, \dots\}$
$n := \text{choose } (k : \text{nat}) : k > 2 \text{ and } k < 4$	$\{3\}$

Result 1 - Background - WhileCC Programming Language

Syntax

Terms

$$t^s ::= x^s \mid F(t_1^{s_1}, \dots, t_m^{s_m})$$

Statements

$S ::=$

skip $\mid \bar{x} := \bar{t} \mid S_1 \ S_2$

$\mid$ if b then S_1 else S_2 fi

$\mid$ while b do S_0 od

$\mid n := \text{choose } (z : \text{nat}) : P(z, \bar{t})$

Procedures

$P ::= \text{proc } D \text{ begin } S \text{ end}$

Algebra $\mathcal{R}$

$$\begin{aligned} 0_{\mathbb{R}}, 1_{\mathbb{R}}, -1_{\mathbb{R}} &: \rightarrow \mathbb{R} \\ +_{\mathbb{R}}, \times_{\mathbb{R}} &: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R} \\ +_{\mathbb{N}}, \times_{\mathbb{N}} &: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N} \\ \text{inv}_{\mathbb{R}} &: \mathbb{R} \rightarrow \mathbb{R} \\ 0_{\mathbb{N}} &: \rightarrow \mathbb{N} \\ \text{suc}_{\mathbb{N}} &: \mathbb{N} \rightarrow \mathbb{N} \\ \text{tt}, \text{ff} &: \rightarrow \mathbb{B} \\ \text{and}, \text{or} &: \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B} \\ \text{not} &: \mathbb{B} \rightarrow \mathbb{B} \\ =_{\mathbb{N}}, <_{\mathbb{N}} &: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{B} \\ =_{\text{real}}, <_{\mathbb{R}} &: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{B} \end{aligned}$$

Semantics

$$\text{inv}_{\mathbb{R}}(x) = \begin{cases} 1/x & \text{if } x \neq 0 \\ \uparrow & \text{otherwise} \end{cases}$$

$$=_{\text{real}}(x, y) = \begin{cases} \text{ff} & \text{if } x \neq y \\ \uparrow & \text{otherwise} \end{cases}$$

$$<_{\mathbb{R}}(x, y) = \begin{cases} \text{tt} & \text{if } x < y \\ \text{ff} & \text{if } x > y \\ \uparrow & \text{if } x = y \end{cases}$$

The Semantics of a program P is denoted by a many-valued function $P^{\mathcal{R}}$.

Statement	Possible Values for n
$n := \text{choose } (k : \text{nat}) : k < 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : \text{toReal}(k) = 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : k < k + 1$	$\{0, 1, 2, \dots\}$
$n := \text{choose } (k : \text{nat}) : k > 2 \text{ and } k < 4$	$\{3\}$

Result 1 - Background - WhileCC Programming Language

Syntax

• Terms

$$t^s ::= x^s \mid F(t_1^{s_1}, \dots, t_m^{s_m})$$

• Statements

$S ::=$

skip $\mid \bar{x} := \bar{t} \mid S_1 \ S_2$

$\mid$ if b then S_1 else S_2 fi

$\mid$ while b do S_0 od

$\mid n := \text{choose } (z : \text{nat}) : P(z, \bar{t})$

• Procedures

$P ::= \text{proc } D \text{ begin } S \text{ end}$

Algebra $\mathcal{R}$

$$\begin{aligned} 0_{\mathbb{R}}, 1_{\mathbb{R}}, -1_{\mathbb{R}} &: \rightarrow \mathbb{R} \\ +_{\mathbb{R}}, \times_{\mathbb{R}} &: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R} \\ +_{\mathbb{N}}, \times_{\mathbb{N}} &: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N} \\ \text{inv}_{\mathbb{R}} &: \mathbb{R} \rightarrow \mathbb{R} \\ 0_{\mathbb{N}} &: \rightarrow \mathbb{N} \\ \text{suc}_{\mathbb{N}} &: \mathbb{N} \rightarrow \mathbb{N} \\ \text{tt}, \text{ff} &: \rightarrow \mathbb{B} \\ \text{and}, \text{or} &: \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B} \\ \text{not} &: \mathbb{B} \rightarrow \mathbb{B} \\ =_{\mathbb{N}}, <_{\mathbb{N}} &: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{B} \\ =_{\text{real}}, <_{\mathbb{R}} &: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{B} \end{aligned}$$

Semantics

$$\text{inv}_{\mathbb{R}}(x) = \begin{cases} 1/x & \text{if } x \neq 0 \\ \uparrow & \text{otherwise} \end{cases}$$

$$=_{\text{real}}(x, y) = \begin{cases} \text{ff} & \text{if } x \neq y \\ \uparrow & \text{otherwise} \end{cases}$$

$$<_{\mathbb{R}}(x, y) = \begin{cases} \text{tt} & \text{if } x < y \\ \text{ff} & \text{if } x > y \\ \uparrow & \text{if } x = y \end{cases}$$

The Semantics of a program P is denoted by a many-valued function $P^{\mathcal{R}}$.

Statement	Possible Values for n
$n := \text{choose } (k : \text{nat}) : k < 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : \text{toReal}(k) = 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : k < k + 1$	$\{0, 1, 2, \dots\}$
$n := \text{choose } (k : \text{nat}) : k > 2 \text{ and } k < 4$	$\{3\}$

Result 1 - Background - WhileCC Programming Language

Syntax

Terms

$$t^s ::= x^s \mid F(t_1^{s_1}, \dots, t_m^{s_m})$$

Statements

$S ::=$

skip $\mid \bar{x} := \bar{t} \mid S_1 \ S_2$

$\mid$ if b then S_1 else S_2 fi

$\mid$ while b do S_0 od

$\mid n := \text{choose} (z : \text{nat}) : P(z, \bar{t})$

Procedures

$P ::= \text{proc } D \text{ begin } S \text{ end}$

Algebra $\mathcal{R}$

$$\begin{aligned} 0_{\mathbb{R}}, 1_{\mathbb{R}}, -1_{\mathbb{R}} &: \rightarrow \mathbb{R} \\ +_{\mathbb{R}}, \times_{\mathbb{R}} &: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R} \\ +_{\mathbb{N}}, \times_{\mathbb{N}} &: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N} \\ \text{inv}_{\mathbb{R}} &: \mathbb{R} \rightarrow \mathbb{R} \\ 0_{\mathbb{N}} &: \rightarrow \mathbb{N} \\ \text{suc}_{\mathbb{N}} &: \mathbb{N} \rightarrow \mathbb{N} \\ \text{tt}, \text{ff} &: \rightarrow \mathbb{B} \\ \text{and}, \text{or} &: \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B} \\ \text{not} &: \mathbb{B} \rightarrow \mathbb{B} \\ =_{\mathbb{N}}, <_{\mathbb{N}} &: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{B} \\ =_{\text{real}}, <_{\mathbb{R}} &: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{B} \end{aligned}$$

Semantics

$$\text{inv}_{\mathbb{R}}(x) = \begin{cases} 1/x & \text{if } x \neq 0 \\ \uparrow & \text{otherwise} \end{cases}$$

$$=_{\text{real}}(x, y) = \begin{cases} \text{ff} & \text{if } x \neq y \\ \uparrow & \text{otherwise} \end{cases}$$

$$<_{\mathbb{R}}(x, y) = \begin{cases} \text{tt} & \text{if } x < y \\ \text{ff} & \text{if } x > y \\ \uparrow & \text{if } x = y \end{cases}$$

The Semantics of a program P is denoted by a many-valued function $P^{\mathcal{R}}$.

Statement	Possible Values for n
$n := \text{choose} (k : \text{nat}) : k < 0$	$\{\uparrow\}$
$n := \text{choose} (k : \text{nat}) : \text{toReal}(k) = 0$	$\{\uparrow\}$
$n := \text{choose} (k : \text{nat}) : k < k + 1$	$\{0, 1, 2, \dots\}$
$n := \text{choose} (k : \text{nat}) : k > 2 \text{ and } k < 4$	$\{3\}$

Result 1 - Background - WhileCC Programming Language

Syntax

Terms

$$t^s ::= x^s \mid F(t_1^{s_1}, \dots, t_m^{s_m})$$

Statements

$S ::=$

skip $\mid \bar{x} := \bar{t} \mid S_1 \ S_2$

$\mid$ if b then S_1 else S_2 fi

$\mid$ while b do S_0 od

$\mid n := \text{choose} (z : \text{nat}) : P(z, \bar{t})$

Procedures

$P ::= \text{proc } D \text{ begin } S \text{ end}$

Algebra $\mathcal{R}$

$0_{\mathbb{R}}, 1_{\mathbb{R}}, -1_{\mathbb{R}} : \rightarrow \mathbb{R}$
 $+_{\mathbb{R}}, \times_{\mathbb{R}} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$
 $+_{\mathbb{N}}, \times_{\mathbb{N}} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$
 $\text{inv}_{\mathbb{R}} : \mathbb{R} \rightarrow \mathbb{R}$
 $0_{\mathbb{N}} : \rightarrow \mathbb{N}$
 $\text{suc}_{\mathbb{N}} : \mathbb{N} \rightarrow \mathbb{N}$
 $\text{tt}, \text{ff} : \rightarrow \mathbb{B}$
 $\text{and}, \text{or} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$
 $\text{not} : \mathbb{B} \rightarrow \mathbb{B}$
 $=_{\mathbb{N}}, <_{\mathbb{N}} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{B}$
 $=_{\text{real}}, <_{\mathbb{R}} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{B}$

Semantics

$$\text{inv}_{\mathbb{R}}(x) = \begin{cases} 1/x & \text{if } x \neq 0 \\ \uparrow & \text{otherwise} \end{cases}$$

$$=_{\text{real}}(x, y) = \begin{cases} \text{ff} & \text{if } x \neq y \\ \uparrow & \text{otherwise} \end{cases}$$

$$<_{\mathbb{R}}(x, y) = \begin{cases} \text{tt} & \text{if } x < y \\ \text{ff} & \text{if } x > y \\ \uparrow & \text{if } x = y \end{cases}$$

The Semantics of a program P is denoted by a many-valued function $P^{\mathcal{R}}$.

Statement	Possible Values for n
$n := \text{choose} (k : \text{nat}) : k < 0$	$\{\uparrow\}$
$n := \text{choose} (k : \text{nat}) : \text{toReal}(k) = 0$	$\{\uparrow\}$
$n := \text{choose} (k : \text{nat}) : k < k + 1$	$\{0, 1, 2, \dots\}$
$n := \text{choose} (k : \text{nat}) : k > 2 \text{ and } k < 4$	$\{3\}$

Result 1 - Background - WhileCC Programming Language

Syntax

• Terms

$$t^s ::= x^s \mid F(t_1^{s_1}, \dots, t_m^{s_m})$$

• Statements

$S ::=$

skip $\mid \bar{x} := \bar{t} \mid S_1 \ S_2$

$\mid$ if b then S_1 else S_2 fi

$\mid$ while b do S_0 od

$\mid n := \text{choose } (z : \text{nat}) : P(z, \bar{t})$

• Procedures

$P ::= \text{proc } D \text{ begin } S \text{ end}$

Algebra $\mathcal{R}$

$0_{\mathbb{R}}, 1_{\mathbb{R}}, -1_{\mathbb{R}} : \rightarrow \mathbb{R}$
 $+_{\mathbb{R}}, \times_{\mathbb{R}} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$
 $+_{\mathbb{N}}, \times_{\mathbb{N}} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$
 $\text{inv}_{\mathbb{R}} : \mathbb{R} \rightarrow \mathbb{R}$
 $0_{\mathbb{N}} : \rightarrow \mathbb{N}$
 $\text{suc}_{\mathbb{N}} : \mathbb{N} \rightarrow \mathbb{N}$
 $\text{tt}, \text{ff} : \rightarrow \mathbb{B}$
 $\text{and}, \text{or} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$
 $\text{not} : \mathbb{B} \rightarrow \mathbb{B}$
 $=_{\mathbb{N}}, <_{\mathbb{N}} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{B}$
 $=_{\text{real}}, <_{\mathbb{R}} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{B}$

Semantics

$$\text{inv}_{\mathbb{R}}(x) = \begin{cases} 1/x & \text{if } x \neq 0 \\ \uparrow & \text{otherwise} \end{cases}$$

$$=_{\text{real}}(x, y) = \begin{cases} \text{ff} & \text{if } x \neq y \\ \uparrow & \text{otherwise} \end{cases}$$

$$<_{\mathbb{R}}(x, y) = \begin{cases} \text{tt} & \text{if } x < y \\ \text{ff} & \text{if } x > y \\ \uparrow & \text{if } x = y \end{cases}$$

The Semantics of a program P is denoted by a many-valued function $P^{\mathcal{R}}$.

Statement	Possible Values for n
$n := \text{choose } (k : \text{nat}) : k < 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : \text{toReal}(k) = 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : k < k + 1$	$\{0, 1, 2, \dots\}$
$n := \text{choose } (k : \text{nat}) : k > 2 \text{ and } k < 4$	$\{3\}$

Result 1 - Background - WhileCC Programming Language

Syntax

Terms

$$t^s ::= x^s \mid F(t_1^{s_1}, \dots, t_m^{s_m})$$

Statements

$S ::=$

skip $\mid \bar{x} := \bar{t} \mid S_1 \ S_2$

$\mid$ if b then S_1 else S_2 fi

$\mid$ while b do S_0 od

$\mid n := \text{choose } (z : \text{nat}) : P(z, \bar{t})$

Procedures

$P ::= \text{proc } D \text{ begin } S \text{ end}$

Algebra $\mathcal{R}$

$0_{\mathbb{R}}, 1_{\mathbb{R}}, -1_{\mathbb{R}} : \rightarrow \mathbb{R}$
 $+_{\mathbb{R}}, \times_{\mathbb{R}} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$
 $+_{\mathbb{N}}, \times_{\mathbb{N}} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$
 $\text{inv}_{\mathbb{R}} : \mathbb{R} \rightarrow \mathbb{R}$
 $0_{\mathbb{N}} : \rightarrow \mathbb{N}$
 $\text{suc}_{\mathbb{N}} : \mathbb{N} \rightarrow \mathbb{N}$
 $\text{tt}, \text{ff} : \rightarrow \mathbb{B}$
 $\text{and}, \text{or} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$
 $\text{not} : \mathbb{B} \rightarrow \mathbb{B}$
 $=_{\mathbb{N}}, <_{\mathbb{N}} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{B}$
 $=_{\text{real}}, <_{\mathbb{R}} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{B}$

Semantics

$$\text{inv}_{\mathbb{R}}(x) = \begin{cases} 1/x & \text{if } x \neq 0 \\ \uparrow & \text{otherwise} \end{cases}$$

$$=_{\text{real}}(x, y) = \begin{cases} \text{ff} & \text{if } x \neq y \\ \uparrow & \text{otherwise} \end{cases}$$

$$<_{\mathbb{R}}(x, y) = \begin{cases} \text{tt} & \text{if } x < y \\ \text{ff} & \text{if } x > y \\ \uparrow & \text{if } x = y \end{cases}$$

The Semantics of a program P is denoted by a many-valued function $P^{\mathcal{R}}$.

Statement	Possible Values for n
$n := \text{choose } (k : \text{nat}) : k < 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : \text{toReal}(k) = 0$	$\{\uparrow\}$
$n := \text{choose } (k : \text{nat}) : k < k + 1$	$\{0, 1, 2, \dots\}$
$n := \text{choose } (k : \text{nat}) : k > 2 \text{ and } k < 4$	$\{3\}$

Result 1 - Background

Definition (**WhileCC**-approximability, [Fu and Zucker, 2014])

A **WhileCC**-procedure P of type $\text{real} \times \text{nat} \rightarrow \text{real}$ on $\mathcal{R}$ is said to *approximate* a function $f : \mathbb{R} \rightarrow \mathbb{R}$ iff for all $n \in \mathbb{N}$ and all $x \in \mathbb{R}$:

- $x \in \text{dom}(f) \implies \uparrow \notin P^{\mathcal{R}}(x, n) \subseteq \text{Nbd}(f(x), 2^{-n})$, and
- $x \notin \text{dom}(f) \implies P^{\mathcal{R}}(x, n) = \{\uparrow\}$

where $\text{Nbd}(y, r)$ has the standard definition of neighborhood on $\mathbb{R}$ i.e.,

$$\text{Nbd}(y, r) = \{z \in \mathbb{R} \mid |y - z| < r\}.$$

Goal

Construct **WhileCC**-procedures approximating elementary functions by induction

Challenge: Using comparison operators introduces undefinedness.

How do we **WhileCC**-approximate “piecewise” functions?

Result 1 - Background

Definition (**WhileCC**-approximability, [Fu and Zucker, 2014])

A **WhileCC**-procedure P of type $\text{real} \times \text{nat} \rightarrow \text{real}$ on $\mathcal{R}$ is said to *approximate* a function $f : \mathbb{R} \rightarrow \mathbb{R}$ iff for all $n \in \mathbb{N}$ and all $x \in \mathbb{R}$:

- $x \in \text{dom}(f) \implies \uparrow \notin P^{\mathcal{R}}(x, n) \subseteq \text{Nbd}(f(x), 2^{-n})$, and
- $x \notin \text{dom}(f) \implies P^{\mathcal{R}}(x, n) = \{\uparrow\}$

where $\text{Nbd}(y, r)$ has the standard definition of neighborhood on $\mathbb{R}$ i.e.,

$$\text{Nbd}(y, r) = \{z \in \mathbb{R} \mid |y - z| < r\}.$$

Goal

Construct **WhileCC**-procedures approximating elementary functions by induction

Challenge: Using comparison operators introduces undefinedness.

How do we **WhileCC**-approximate “piecewise” functions?

Result 1 - Background

Definition (**WhileCC**-approximability, [Fu and Zucker, 2014])

A **WhileCC**-procedure P of type $\text{real} \times \text{nat} \rightarrow \text{real}$ on $\mathcal{R}$ is said to *approximate* a function $f : \mathbb{R} \rightarrow \mathbb{R}$ iff for all $n \in \mathbb{N}$ and all $x \in \mathbb{R}$:

- $x \in \text{dom}(f) \implies \uparrow \notin P^{\mathcal{R}}(x, n) \subseteq \text{Nbd}(f(x), 2^{-n})$, and
- $x \notin \text{dom}(f) \implies P^{\mathcal{R}}(x, n) = \{\uparrow\}$

where $\text{Nbd}(y, r)$ has the standard definition of neighborhood on $\mathbb{R}$ i.e.,

$$\text{Nbd}(y, r) = \{z \in \mathbb{R} \mid |y - z| < r\}.$$

Goal

Construct **WhileCC**-procedures approximating elementary functions by induction

Challenge: Using comparison operators introduces undefinedness.

How do we **WhileCC**-approximate “piecewise” functions?

Result 1 - Background

Definition (**WhileCC**-approximability, [Fu and Zucker, 2014])

A **WhileCC**-procedure P of type $\text{real} \times \text{nat} \rightarrow \text{real}$ on $\mathcal{R}$ is said to *approximate* a function $f : \mathbb{R} \rightarrow \mathbb{R}$ iff for all $n \in \mathbb{N}$ and all $x \in \mathbb{R}$:

- $x \in \text{dom}(f) \implies \uparrow \notin P^{\mathcal{R}}(x, n) \subseteq \text{Nbd}(f(x), 2^{-n})$, and
- $x \notin \text{dom}(f) \implies P^{\mathcal{R}}(x, n) = \{\uparrow\}$

where $\text{Nbd}(y, r)$ has the standard definition of neighborhood on $\mathbb{R}$ i.e.,

$$\text{Nbd}(y, r) = \{z \in \mathbb{R} \mid |y - z| < r\}.$$

Goal

Construct **WhileCC**-procedures approximating elementary functions by induction

Challenge: Using comparison operators introduces undefinedness.

How do we **WhileCC**-approximate “piecewise” functions?

Result 1 - Background

Definition (**WhileCC**-approximability, [Fu and Zucker, 2014])

A **WhileCC**-procedure P of type $\text{real} \times \text{nat} \rightarrow \text{real}$ on $\mathcal{R}$ is said to *approximate* a function $f : \mathbb{R} \rightarrow \mathbb{R}$ iff for all $n \in \mathbb{N}$ and all $x \in \mathbb{R}$:

- $x \in \text{dom}(f) \implies \uparrow \notin P^{\mathcal{R}}(x, n) \subseteq \text{Nbd}(f(x), 2^{-n})$, and
- $x \notin \text{dom}(f) \implies P^{\mathcal{R}}(x, n) = \{\uparrow\}$

where $\text{Nbd}(y, r)$ has the standard definition of neighborhood on $\mathbb{R}$ i.e.,

$$\text{Nbd}(y, r) = \{z \in \mathbb{R} \mid |y - z| < r\}.$$

Goal

Construct **WhileCC**-procedures approximating elementary functions by induction

Challenge: Using comparison operators introduces undefinedness.

How do we **WhileCC**-approximate “piecewise” functions?

Result 1 - Background

Definition (**WhileCC**-approximability, [Fu and Zucker, 2014])

A **WhileCC**-procedure P of type $\text{real} \times \text{nat} \rightarrow \text{real}$ on $\mathcal{R}$ is said to *approximate* a function $f : \mathbb{R} \rightarrow \mathbb{R}$ iff for all $n \in \mathbb{N}$ and all $x \in \mathbb{R}$:

- $x \in \text{dom}(f) \implies \uparrow \notin P^{\mathcal{R}}(x, n) \subseteq \text{Nbd}(f(x), 2^{-n})$, and
- $x \notin \text{dom}(f) \implies P^{\mathcal{R}}(x, n) = \{\uparrow\}$

where $\text{Nbd}(y, r)$ has the standard definition of neighborhood on $\mathbb{R}$ i.e.,

$$\text{Nbd}(y, r) = \{z \in \mathbb{R} \mid |y - z| < r\}.$$

Goal

Construct **WhileCC**-procedures approximating elementary functions by induction

Challenge: Using comparison operators introduces undefinedness.

How do we **WhileCC**-approximate “piecewise” functions?

Result 1 - Background

Definition (**WhileCC**-approximability, [Fu and Zucker, 2014])

A **WhileCC**-procedure P of type $\text{real} \times \text{nat} \rightarrow \text{real}$ on $\mathcal{R}$ is said to *approximate* a function $f : \mathbb{R} \rightarrow \mathbb{R}$ iff for all $n \in \mathbb{N}$ and all $x \in \mathbb{R}$:

- $x \in \text{dom}(f) \implies \uparrow \notin P^{\mathcal{R}}(x, n) \subseteq \text{Nbd}(f(x), 2^{-n})$, and
- $x \notin \text{dom}(f) \implies P^{\mathcal{R}}(x, n) = \{\uparrow\}$

where $\text{Nbd}(y, r)$ has the standard definition of neighborhood on $\mathbb{R}$ i.e.,

$$\text{Nbd}(y, r) = \{z \in \mathbb{R} \mid |y - z| < r\}.$$

Goal

Construct **WhileCC**-procedures approximating elementary functions by induction

Challenge: Using comparison operators introduces undefinedness.

How do we **WhileCC**-approximate “piecewise” functions?

Result 1 - Background

Definition (**WhileCC**-approximability, [Fu and Zucker, 2014])

A **WhileCC**-procedure P of type $\text{real} \times \text{nat} \rightarrow \text{real}$ on $\mathcal{R}$ is said to *approximate* a function $f : \mathbb{R} \rightarrow \mathbb{R}$ iff for all $n \in \mathbb{N}$ and all $x \in \mathbb{R}$:

- $x \in \text{dom}(f) \implies \uparrow \notin P^{\mathcal{R}}(x, n) \subseteq \text{Nbd}(f(x), 2^{-n})$, and
- $x \notin \text{dom}(f) \implies P^{\mathcal{R}}(x, n) = \{\uparrow\}$

where $\text{Nbd}(y, r)$ has the standard definition of neighborhood on $\mathbb{R}$ i.e.,

$$\text{Nbd}(y, r) = \{z \in \mathbb{R} \mid |y - z| < r\}.$$

Goal

Construct **WhileCC**-procedures approximating elementary functions by induction

Challenge: Using comparison operators introduces undefinedness.

How do we **WhileCC**-approximate “piecewise” functions?

Result 1 - Background

Definition (**WhileCC**-approximability, [Fu and Zucker, 2014])

A **WhileCC**-procedure P of type $\text{real} \times \text{nat} \rightarrow \text{real}$ on $\mathcal{R}$ is said to *approximate* a function $f : \mathbb{R} \rightarrow \mathbb{R}$ iff for all $n \in \mathbb{N}$ and all $x \in \mathbb{R}$:

- $x \in \text{dom}(f) \implies \uparrow \notin P^{\mathcal{R}}(x, n) \subseteq \text{Nbd}(f(x), 2^{-n})$, and
- $x \notin \text{dom}(f) \implies P^{\mathcal{R}}(x, n) = \{\uparrow\}$

where $\text{Nbd}(y, r)$ has the standard definition of neighborhood on $\mathbb{R}$ i.e.,

$$\text{Nbd}(y, r) = \{z \in \mathbb{R} \mid |y - z| < r\}.$$

Goal

Construct **WhileCC**-procedures approximating elementary functions by induction

Challenge: Using comparison operators introduces undefinedness.

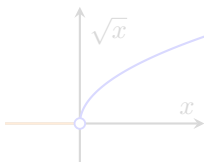
How do we **WhileCC**-approximate “piecewise” functions?

Result 1 - Challenges

Problem

When defining piecewise functions, comparison makes a hole!

Example: Even Root - First Attempt



Solution

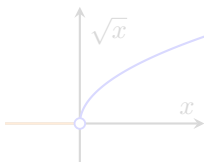
- Find an overlapping interval where both pieces are defined and give a good enough answer
- Use the nondeterminism of “choose”

Result 1 - Challenges

Problem

When defining piecewise functions, comparison makes a hole!

Example: Even Root - First Attempt



Solution

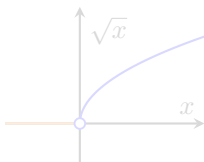
- Find an overlapping interval where both pieces are defined and give a good enough answer
- Use the nondeterminism of “choose”

Result 1 - Challenges

Problem

When defining piecewise functions, comparison makes a hole!

Example: Even Root - First Attempt



Solution

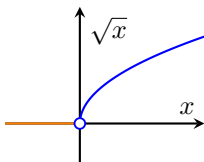
- Find an overlapping interval where both pieces are defined and give a good enough answer
- Use the nondeterminism of “choose”

Result 1 - Challenges

Problem

When defining piecewise functions, comparison makes a hole!

Example: Even Root - First Attempt



Solution

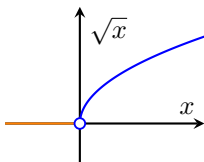
- Find an overlapping interval where both pieces are defined and give a good enough answer
- Use the nondeterminism of “choose”

Result 1 - Challenges

Problem

When defining piecewise functions, comparison makes a hole!

Example: Even Root - First Attempt



Solution

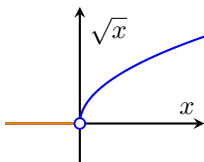
- Find an overlapping interval where both pieces are defined and give a good enough answer
- Use the nondeterminism of “choose”

Result 1 - Challenges

Problem

When defining piecewise functions, comparison makes a hole!

Example: Even Root - First Attempt



Solution

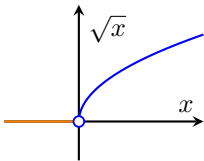
- Find an overlapping interval where both pieces are defined and give a good enough answer
- Use the nondeterminism of “choose”

Result 1 - Challenges

Problem

When defining piecewise functions, comparison makes a hole!

Example: Even Root - First Attempt



Solution

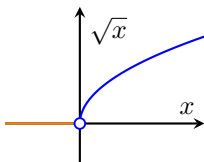
- Find an overlapping interval where both pieces are defined and give a good enough answer
- Use the nondeterminism of “choose”

Result 1 - Challenges

Problem

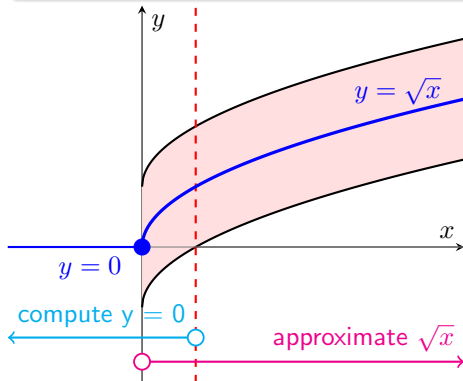
When defining piecewise functions, comparison makes a hole!

Example: Even Root - First Attempt



Solution

- Find an overlapping interval where both pieces are defined and give a good enough answer
- Use the nondeterminism of “choose”



Result 2

Theorem 2.2 (Acceptability Theorem: Part 2)

Any elementary function is effectively locally uniformly continuous **w.r.t. an effective open exhaustion** for its domain.

(We prove that this is) equivalent to proving:

Any elementary function has a local modulus of continuity.

Definition (Local modulus of continuity)

Let $f : \mathbb{R} \rightarrow \mathbb{R}$. A recursive function $N : \mathbb{Q} \times \mathbb{Q} \times \mathbb{N} \rightarrow \mathbb{N}$ is called a **local modulus of continuity** for f iff for any $a, b \in \mathbb{Q}$ with $[a, b] \subseteq \text{dom}(f)$ and $k \in \mathbb{N}$, we have

$$\forall x, y \in (a, b) \quad |x - y| < 2^{-N(a,b,k)} \implies |f(x) - f(y)| < 2^{-k}.$$

The notion of effective local uniform continuity is independent of effective open exhaustion.

Result 2

Theorem 2.2 (Acceptability Theorem: Part 2)

Any elementary function is effectively locally uniformly continuous **w.r.t. an effective open exhaustion** for its domain.

(We prove that this is) equivalent to proving:

Any elementary function has a local modulus of continuity.

Definition (Local modulus of continuity)

Let $f : \mathbb{R} \rightarrow \mathbb{R}$. A recursive function $N : \mathbb{Q} \times \mathbb{Q} \times \mathbb{N} \rightarrow \mathbb{N}$ is called a **local modulus of continuity** for f iff for any $a, b \in \mathbb{Q}$ with $[a, b] \subseteq \text{dom}(f)$ and $k \in \mathbb{N}$, we have

$$\forall x, y \in (a, b) \quad |x - y| < 2^{-N(a, b, k)} \implies |f(x) - f(y)| < 2^{-k}.$$

The notion of effective local uniform continuity is independent of effective open exhaustion.

Result 2

Theorem 2.2 (Acceptability Theorem: Part 2)

Any elementary function is effectively locally uniformly continuous **w.r.t. an effective open exhaustion** for its domain.

(We prove that this is) equivalent to proving:

Any elementary function has a local modulus of continuity.

Definition (Local modulus of continuity)

Let $f : \mathbb{R} \rightarrow \mathbb{R}$. A recursive function $N : \mathbb{Q} \times \mathbb{Q} \times \mathbb{N} \rightarrow \mathbb{N}$ is called a **local modulus of continuity** for f iff for any $a, b \in \mathbb{Q}$ with $[a, b] \subseteq \text{dom}(f)$ and $k \in \mathbb{N}$, we have

$$\forall x, y \in (a, b) \quad |x - y| < 2^{-N(a, b, k)} \implies |f(x) - f(y)| < 2^{-k}.$$

The notion of effective local uniform continuity is independent of effective open exhaustion.

Result 2

Theorem 2.2 (Acceptability Theorem: Part 2)

Any elementary function is effectively locally uniformly continuous **w.r.t. an effective open exhaustion** for its domain.

(We prove that this is) equivalent to proving:

Any elementary function has a local modulus of continuity.

Definition (Local modulus of continuity)

Let $f : \mathbb{R} \rightarrow \mathbb{R}$. A recursive function $N : \mathbb{Q} \times \mathbb{Q} \times \mathbb{N} \rightarrow \mathbb{N}$ is called a **local modulus of continuity** for f iff for any $a, b \in \mathbb{Q}$ with $[a, b] \subseteq \text{dom}(f)$ and $k \in \mathbb{N}$, we have

$$\forall x, y \in (a, b) \quad |x - y| < 2^{-N(a, b, k)} \implies |f(x) - f(y)| < 2^{-k}.$$

The notion of effective local uniform continuity is independent of effective open exhaustion.

Result 2

Theorem 2.2 (Acceptability Theorem: Part 2)

Any elementary function is effectively locally uniformly continuous **w.r.t. an effective open exhaustion** for its domain.

(We prove that this is) equivalent to proving:

Any elementary function has a local modulus of continuity.

Definition (Local modulus of continuity)

Let $f : \mathbb{R} \rightarrow \mathbb{R}$. A recursive function $N : \mathbb{Q} \times \mathbb{Q} \times \mathbb{N} \rightarrow \mathbb{N}$ is called a **local modulus of continuity** for f iff for any $a, b \in \mathbb{Q}$ with $[a, b] \subseteq \text{dom}(f)$ and $k \in \mathbb{N}$, we have

$$\forall x, y \in (a, b) \quad |x - y| < 2^{-N(a, b, k)} \implies |f(x) - f(y)| < 2^{-k}.$$

The notion of effective local uniform continuity is independent of effective open exhaustion.

Summary

We proved that:

- all elementary functions are **WhileCC**-approximable.
- all elementary functions are acceptable.

We also

- presented an **alternative characterization** of acceptable functions using the **local modulus of continuity** concept.

Summary

We proved that:

- all elementary functions are **WhileCC**-approximable.
- all elementary functions are acceptable.

We also

- presented an **alternative characterization** of acceptable functions using the **local modulus of continuity** concept.

Summary

We proved that:

- all elementary functions are **WhileCC**-approximable.
- all elementary functions are acceptable.

We also

- presented an **alternative characterization** of acceptable functions using the **local modulus of continuity** concept.

Summary

We proved that:

- all elementary functions are **WhileCC**-approximable.
- all elementary functions are acceptable.

We also

- presented an **alternative characterization** of acceptable functions using the **local modulus of continuity** concept.

Summary

We proved that:

- all elementary functions are **WhileCC**-approximable.
- all elementary functions are acceptable.

We also

- presented an **alternative characterization** of acceptable functions using the **local modulus of continuity** concept.

Alternative Approach

Theorem

The following are true for any function f on the reals:

- f is GL-computable $\Rightarrow f$ is effectively uniformly continuous w.r.t. any compact set [Anonymous Reviewer].
- f is GL-computable + has an r.e. open domain $\Rightarrow f$ is effectively uniformly continuous [Pauly, 2016].

Theorem

Elementary functions are GL-computable and have open domains.

Proof.

Straightforward proof by induction. □

Corollary

Elementary functions are acceptable.

Alternative Approach

Theorem

The following are true for any function f on the reals:

- f is GL-computable $\Rightarrow f$ is effectively uniformly continuous w.r.t. any compact set [Anonymous Reviewer].
- f is GL-computable + has an r.e. open domain $\Rightarrow f$ is effectively uniformly continuous [Pauly, 2016].

Theorem

Elementary functions are GL-computable and have open domains.

Proof.

Straightforward proof by induction. □

Corollary

Elementary functions are acceptable.

Alternative Approach

Theorem

The following are true for any function f on the reals:

- f is GL-computable $\Rightarrow f$ is effectively uniformly continuous w.r.t. any compact set [Anonymous Reviewer].
- f is GL-computable + has an r.e. open domain $\Rightarrow f$ is effectively uniformly continuous [Pauly, 2016].

Theorem

Elementary functions are GL-computable and have open domains.

Proof.

Straightforward proof by induction. □

Corollary

Elementary functions are acceptable.

Alternative Approach

Theorem

The following are true for any function f on the reals:

- f is GL-computable $\Rightarrow f$ is effectively uniformly continuous w.r.t. any compact set [Anonymous Reviewer].
- f is GL-computable + has an r.e. open domain $\Rightarrow f$ is effectively uniformly continuous [Pauly, 2016].

Theorem

Elementary functions are GL-computable and have open domains.

Proof.

Straightforward proof by induction. □

Corollary

Elementary functions are acceptable.

Alternative Approach

Theorem

The following are true for any function f on the reals:

- f is GL-computable $\Rightarrow f$ is effectively uniformly continuous w.r.t. any compact set [Anonymous Reviewer].
- f is GL-computable + has an r.e. open domain $\Rightarrow f$ is effectively uniformly continuous [Pauly, 2016].

Theorem

Elementary functions are GL-computable and have open domains.

Proof.

Straightforward proof by induction. □

Corollary

Elementary functions are acceptable.

Alternative Approach

Theorem

The following are true for any function f on the reals:

- f is GL-computable $\Rightarrow f$ is effectively uniformly continuous w.r.t. any compact set [Anonymous Reviewer].
- f is GL-computable + has an r.e. open domain $\Rightarrow f$ is effectively uniformly continuous [Pauly, 2016].

Theorem

Elementary functions are GL-computable and have open domains.

Proof.

Straightforward proof by induction. □

Corollary

Elementary functions are acceptable.

Future Work

Conjecture:

- All partial unary **WhileCC**-approximable functions are acceptable.
 - **If the conjecture holds**, are *non-unary* **WhileCC**-approximable functions acceptable?
 - **If not**, what is a model of computation that characterizes exactly the class of acceptable functions?

Also, we would like to know if acceptability is the weakest condition that can be used to prove the equivalence theorem.

Conjecture:

- All partial unary **WhileCC**-approximable functions are acceptable.
 - If the conjecture holds, are *non-unary* **WhileCC**-approximable functions acceptable?
 - If not, what is a model of computation that characterizes exactly the class of acceptable functions?

Also, we would like to know if acceptability is the weakest condition that can be used to prove the equivalence theorem.

Conjecture:

- All partial unary **WhileCC**-approximable functions are acceptable.
 - **If the conjecture holds**, are *non-unary* **WhileCC**-approximable functions acceptable?
 - **If not**, what is a model of computation that characterizes exactly the class of acceptable functions?

Also, we would like to know if acceptability is the weakest condition that can be used to prove the equivalence theorem.

Conjecture:

- All partial unary **WhileCC**-approximable functions are acceptable.
 - **If the conjecture holds**, are *non-unary* **WhileCC**-approximable functions acceptable?
 - **If not**, what is a model of computation that characterizes exactly the class of acceptable functions?

Also, we would like to know if acceptability is the weakest condition that can be used to prove the equivalence theorem.

Conjecture:

- All partial unary **WhileCC**-approximable functions are acceptable.
 - **If the conjecture holds**, are *non-unary* **WhileCC**-approximable functions acceptable?
 - **If not**, what is a model of computation that characterizes exactly the class of acceptable functions?

Also, we would like to know if acceptability is the weakest condition that can be used to prove the equivalence theorem.

References

- M. Q. Fu and J. Zucker. Models of computation for partial functions on the reals. *Journal of Logical and Algebraic Methods in Programming*, 84(2):218–237, 11 2014. ISSN 2352-2208.
- A. Pauly. On the topological aspects of the theory of represented spaces. *Computability*, 5:159–180, 05 2016. doi:[10.3233/COM-150049](https://doi.org/10.3233/COM-150049).
- M. Tenenbaum and H. Pollard. *Ordinary Differential Equations: An Elementary Textbook for Students of Mathematics, Engineering, and the Sciences*. Dover Books on Mathematics. Dover Publications, 1985. ISBN 9780486649405. page 17.
- K. Weihrauch. *Computable Analysis - An Introduction*. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2000. ISBN 978-3-540-66817-6.

A HUGE Thank you!